

Fulton Hogan Assignment

Software Modelling FINAL Report ENSE706

Fateh Bhular 23217987



Summary from Progress Report

The first phase of this assignment focused on the initial planning of the new Project Costing & Management system of Fulton Hogan. We focused on the three most important sub-departments which were **Project Management, Finance, and Operations**. Each of these departments have three important stakeholders each:



Project Management	Finance	Operations
Project Manager Project Coordinator Site Lead	Financial Controller Auditor Project Accountant	Site Foreman Heavy-Machine Operator General Labourer

During requirements elicitation, two stakeholders from each sub-department were interviewed about their current system “JD Edwards”. It was noted that each user faced issues that reduced productivity or halted work all together. After all the interviews were finished, it was determined that their existing system suffered from data latency, a lack of real-time synchronisation, and data inconsistencies.

With the interviews, and some observations of JD Edwards online, the results were translated into functional and non-function requirements. These requirements were analysed thoroughly and focused on the main issues of the existing system being data integrity, system consistency, and communication between/in departments.

During UML Requirements Modelling, the system’s behaviour was implemented using a **Use-Case Diagram**. This diagram mapped the relationships between actors-actions, and actions-actions. The final iteration of the user-case diagram was used as a blueprint for the **Class Diagram** later on, which focused on the structural frame of the system program. This diagram converted the different use-cases into concrete and abstract classes with labeled interactions. The last iteration of the class diagram was a great improvement over the first iteration, as more classes were added to support previous inconsistencies. Design principles such as SRP, ISP, etc were improved throughout the iterations, and abstract classes were also improved.

During the iterations of the use-case and class diagrams, the **C# prototype** was being produced. The initial iteration was focused on the main concrete classes that were stated in the use-case descriptions section. During production, design risks were identified, such as high coupling, low cohesion, not following SRP, and more. The Agile cycle was implemented because it was the right fit, allowing improvements to be made during development and not at the end when everything is almost finished.

This class diagram also showcased **high coupling** and **low cohesion** between classes. This means that some classes were very dependent on other classes, while some classes had more than one responsibility. These are very severe system problems that become harder to handle the more you scale and maintain the system. With high coupling, modifying a class will affect

the other classes that are dependent on it, and make you spend more time ensuring your changes work at runtime. A critical part of the system is the *Project* class, this class has attributes that can change at runtime. Classes like *Task*, *TaskList*, *Instructions*, and some report classes are tightly coupled to the *Project* class, meaning that changes in the *Project* class will affect all the mentioned classes (*Figure 1*). This makes the system very vulnerable to system errors because changes in one class affect several others.

SOLID principles have been focused on, however there are some ends of the diagram that need polishing. *GeneralLabourer* and *HeavyMachineOperator* have the system login and logout methods that are inherited from the *User* class. This violates the **Liskov Substitution principle** because those two classes inherit from *User* but have different behaviours. Those two sub-classes are employees who never use the main system, so substituting them in-place of *User* could cause a system error. The **Open/Closed principle** is being violated by the *User* class. Because *User* is a concrete class, it has implementations, and the methods are defined. If we wanted to add a new type of employee to the system, *User* would have to be modified, thus violating OCP.

Another noticeable issue with the diagram in *Figure 1* is that the interfaces don't follow proper structure. Interfaces are meant to define behaviour of classes that implement them. In the diagram however, it stores a boolean attribute and no methods.

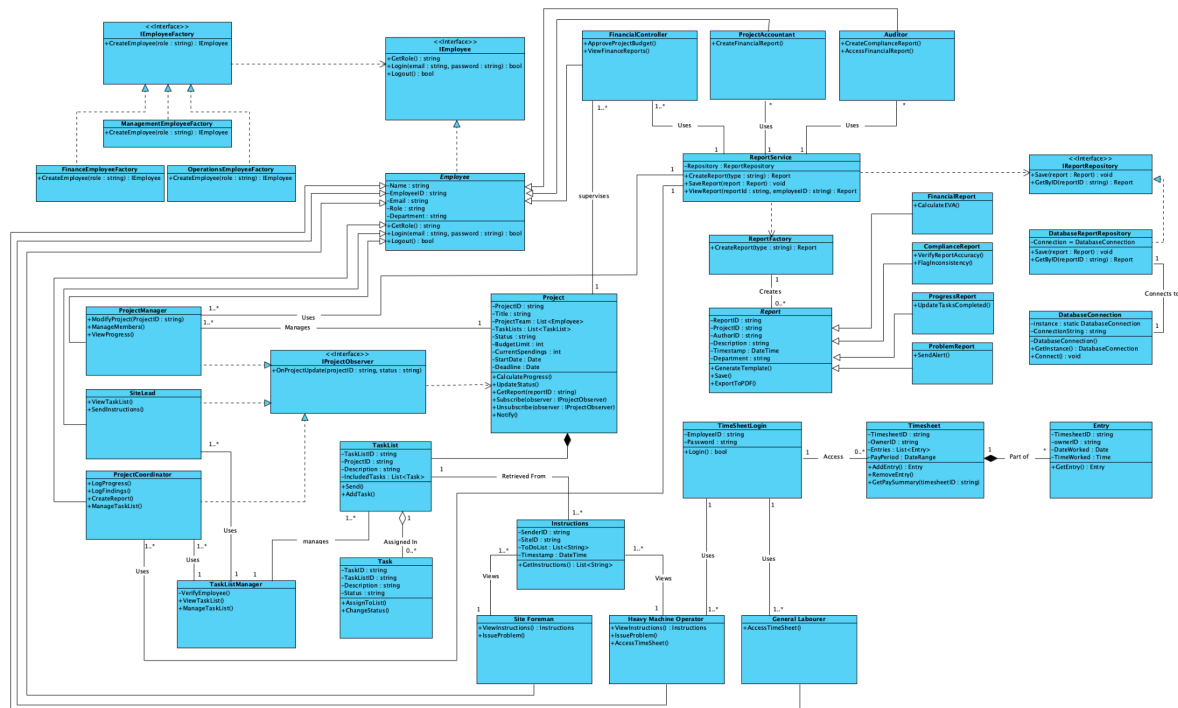


Figure 2: Refined UML Class Diagram

SOLID. The *ReportService* and *ReportFactory* classes (Figure 2) are the result of splitting the legacy *Report* concrete class. Now every employee accesses *ReportService* which is responsible for processing reports, and that can access *ReportFactory* which creates reports. This solves the SRP violation in the legacy class diagram, where data processing and creation was cluttered into a single concrete *Report* class. The new *ReportService* and *ReportFactory* also follows the **high cohesion** principle that is very important in software system design. The relationships between different roles and objects have been decoupled using mediator objects, e.g. *TaskListManager* and *ReportService*. *ProjectCoordinator* and *SiteLead* now interact with *TaskListManager* instead of directly to *TaskList*, which decreases the coupling between the two classes following **low coupling** principle. This allows for modifications to be made in an isolated environment, so that other classes are not affected by the new changes.

1.2 Sequence Diagrams

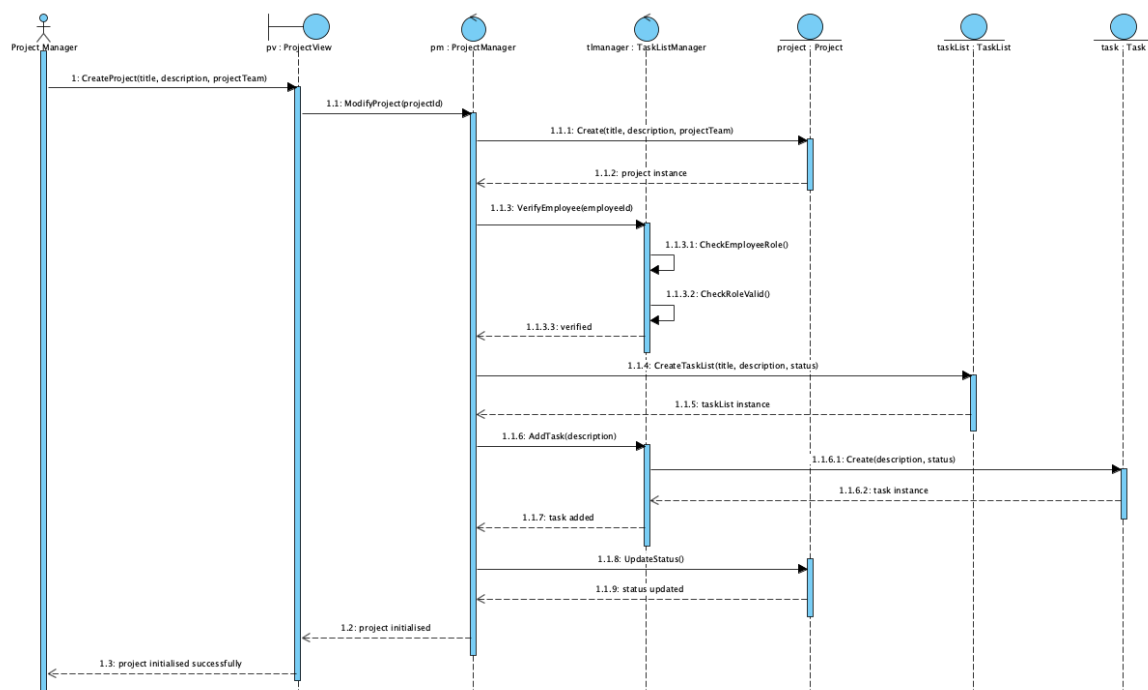


Figure 4: Sequence Diagram for “Modifying a project”

The sequence diagram in Figure 4 models a *ProjectManager* employee **modifying a project**. This diagram uses the Boundary-Control-Entity pattern, where all three lifelines have a responsibility in this sequence of actions. The *ProjectManager* actor interacts with the *ProjectView* boundary, which separates the complex, behind-the-scenes logic from the user. There is a *ProjectManager* control lifeline which is the primary controller. Responsibility of project modification is split between *ProjectManager* which creates the project, and *TaskListManager*, which handles task-related jobs.

Before any important modifications are made, the *ProjectManager* control lifeline runs self-messages to check if the user has the correct privileges. This is very important for security reasons so that someone at a lower level cannot change something crucial about the project.

This sequence diagram handles the instantiation of three objects being *Project*, *TaskList*, and *Task*. The *ProjectManager* control entity creates a project, then gives control to *TaskListManager* which creates the task list and tasks. Once the instantiation is complete, the messages flow back up through the lifelines, and at the end, a message “project initialised successfully” confirms that each lifeline completed its responsibility. The final result is reported to the *ProjectManager* actor via the *ProjectView* boundary.

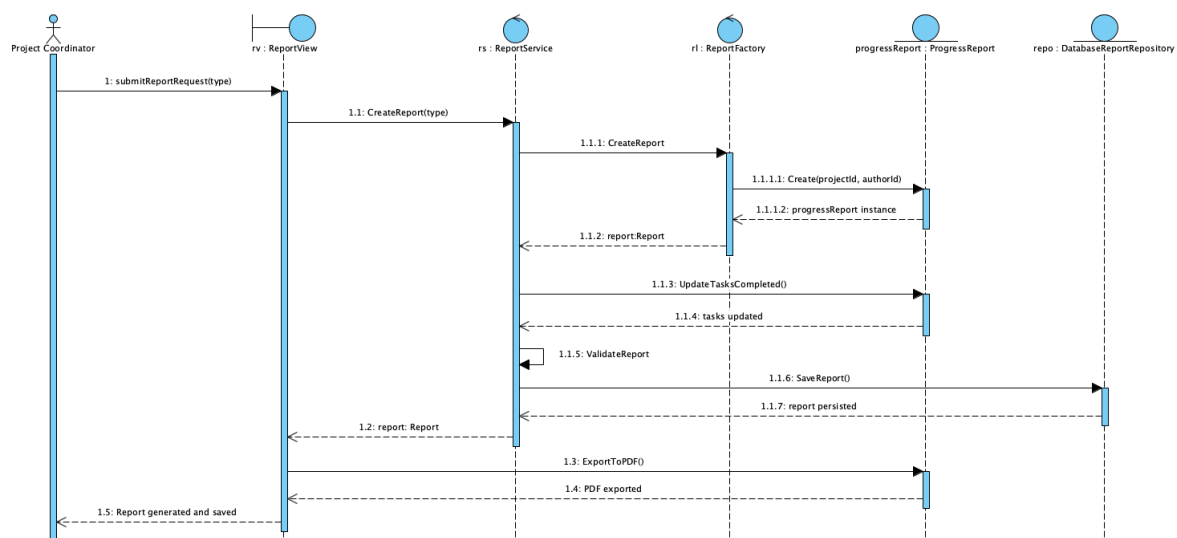


Figure 5: Sequence Diagram for “Generating a Progress Report”

The sequence diagram in *Figure 5* models a *ProjectCoordinator* **generating a progress report**. This diagram also uses the Boundary-Control-Entity pattern, where *ReportView* acts as a boundary separating the *ProjectCoordinator* from the complex system logic. *ReportService* is the primary controller and *ProgressReport* and *DatabaseReportRepository* are the entities.

A design choice in this diagram was to add *ReportFactory*. Instead of *ReportService* being responsible for both data handling and creation, the responsibility is now split up into two classes which follow the **Single Responsibility Principle**. The factory handles the creation call and returns an instance back to the service, which keeps object creation logic separate from business logic.

This sequence diagram handles the instantiation of a *ProgressReport* entity. Because *ReportService* is responsible for business logic, the creation is handled by *ReportFactory*,

which returns a *ProgressReport* instance. After creation, *ReportService* uses methods to modify the progress report and save it to *DatabaseReportRepository*. After the report has been exported as a PDF, the messages go back up through the lifelines, and a message “Report generated and saved” confirms the completion of this sequence.

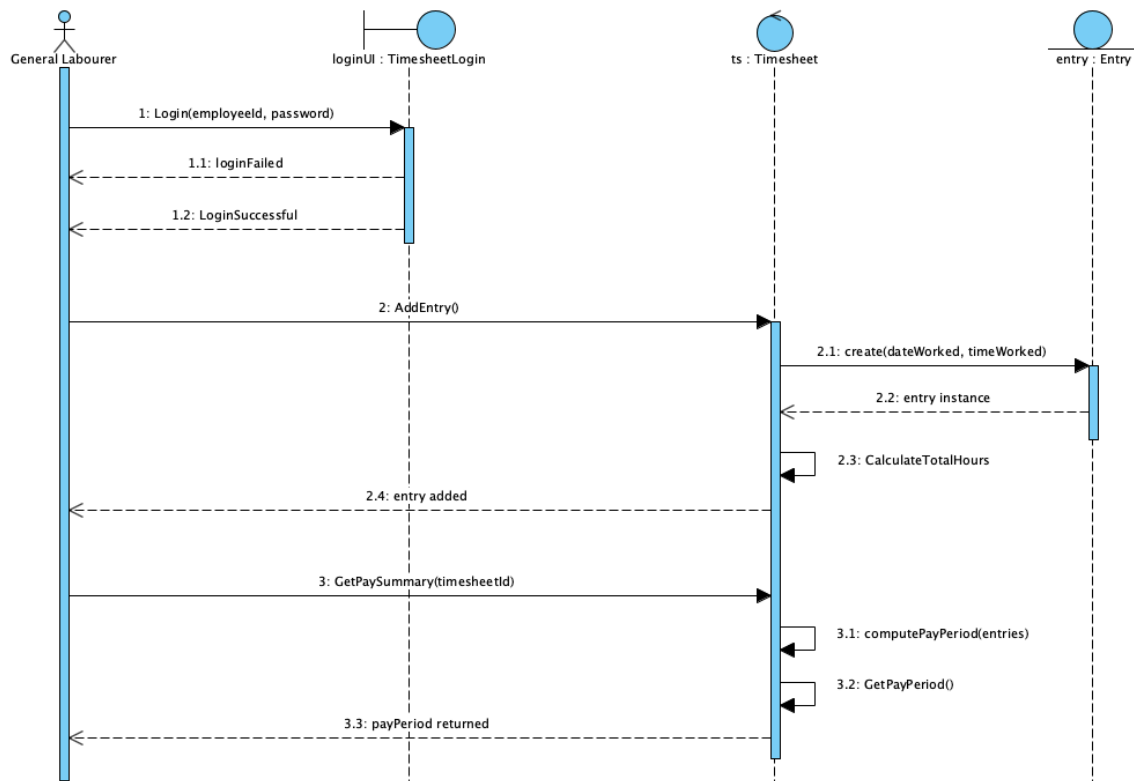


Figure 6: Sequence Diagram for “Creating a timesheet entry”

The sequence diagram in *Figure 6* models a *GeneralLabourer* **creating a timesheet entry** on the timesheet system. This diagram uses a Boundary-Control-Entity pattern, where all three lifelines have a responsibility. The *GeneralLabourer* actor interacts with the *TimesheetLogin* boundary which separates the system logic from them. The *Timesheet* control entity is the primary controller that handles timesheet operations. Responsibility of creating an entry is split between *Timesheet* and *Entry* where *Timesheet* manages overall timesheet logic, and *Entry* handles the creation of entries.

Before any entries are made, the *TimesheetLogin* boundary validates the *GeneralLabourer*’s credentials before allowing them into the timesheet system. If the login succeeds, then the *GeneralLabourer* will be able to add an entry and get pay summaries. If the login fails, they will be told that their credentials are incorrect. This was implemented for the purpose of security, as we don’t want an outsider logging into an employee’s timesheet.

This sequence diagram handles the instantiation of the *Entry* entity using the *create()* method from *Timesheet*. Once the entry is added, the *GeneralLabourer* can request a pay summary which triggers two self-messages (1) computing the payperiod using current entries (2) gets the pay period. Instead of making the *Entry* entity handle logic, *Timesheet* handles the logic since it is the controller. The return messages flow back through the lifelines, with the message “payPeriod returned” confirming that the sequence is complete.

1.3 Activity Diagrams

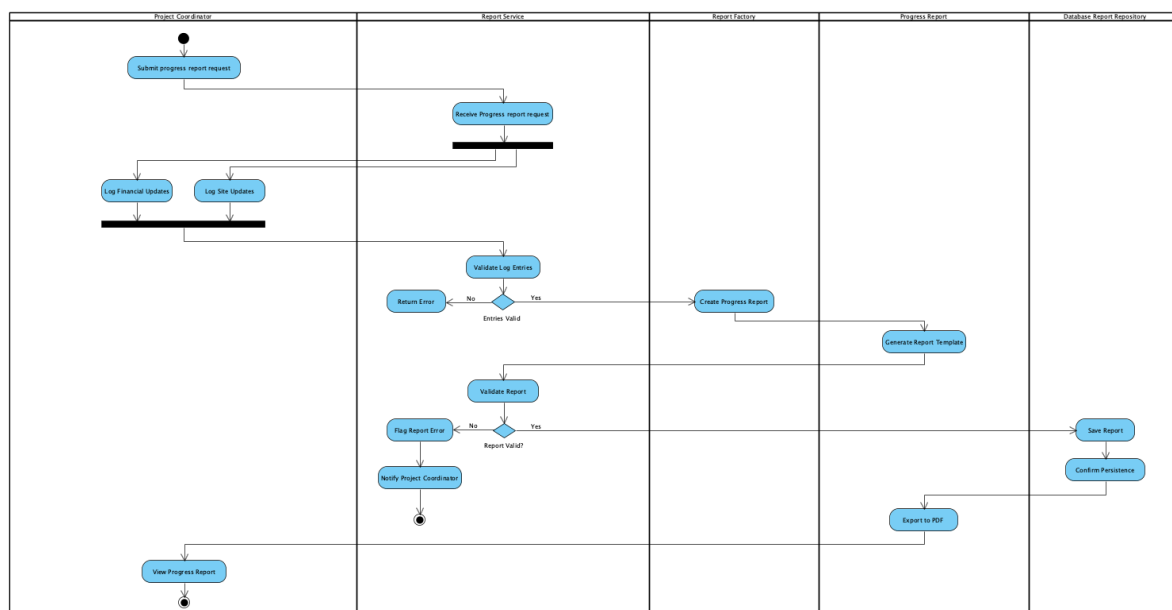


Figure 7: Activity Diagram for “Logging Progress and Generating Report”

The activity diagram in *Figure 7* models the progress report generation workflow using five different swimlanes: *ProjectCoordinator*, *ReportService*, *ReportFactory*, *ProgressReport*, and *DatabaseReportRepository*. Each of these swimlanes has a distinct responsibility following the **Single Responsibility Principle**, which makes sure that no class alone is controlling everything. The different swimlanes represents the low coupling between the classes, e.g. *ReportService* gives the task of report creation to *ReportFactory*, and lets them handle everything. This is an implementation of the **Simple Factory Method**, which is a stepping stone to the more known **Factory Method**.

Fork and **join** nodes were used in the diagram to showcase concurrency during the logging. The *ProjectCoordinator* can choose to log financial or site updates into the report, and neither action depends on the other. The two threads are then joined, which forces both tasks to be done before the report verification can occur. Because the *ProjectCoordinator* is responsible

for entering data, and the *ReportServer* handles the validation and submission, this demonstrates **high cohesion** in the classes.

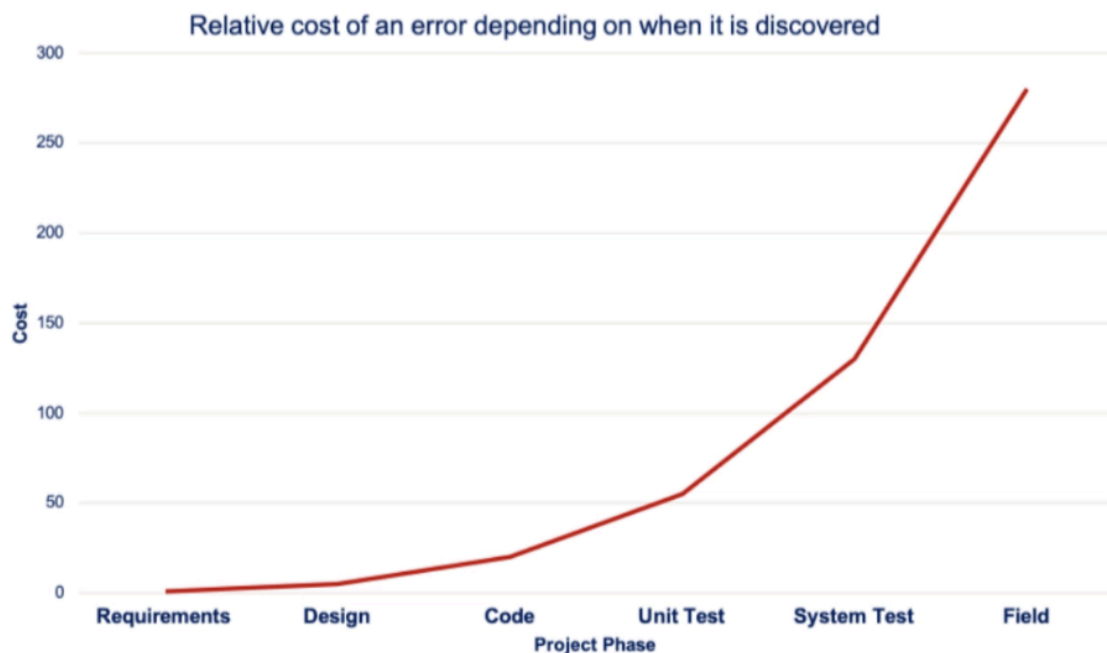


Figure 8: Boehm’s First Law

There are two **decision nodes** that are used in *ReportService*, both implementing guard conditions. The first guard condition makes sure that the validation of log entries succeeds. If it is a success, then the *ReportFactory* will create the progress report, otherwise an error is returned. The second guard condition checks to see if the report can be stored in the database. This guard condition makes sure the report meets the criteria for what a report must contain and avoid. If it succeeds, then it will be stored, otherwise the *ProjectCoordinator* is notified that “There was an error persisting in this report”. Both guard conditions play the role of security, making sure no invalid data is passed through the swimlanes which would cause an error later on. *Figure 7* was built with **Boehm’s first law** as its foundation (*Figure 8*), “Errors are most frequent during requirements and design activities, and are more expensive the later they are removed” - Boehm.

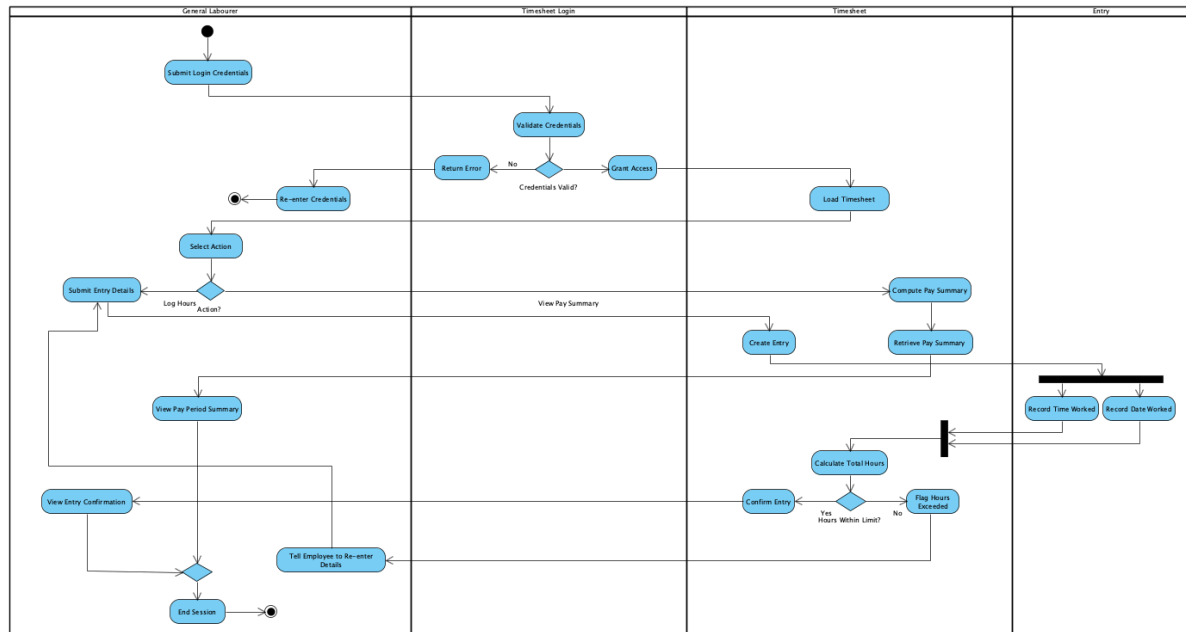


Figure 9: Activity Diagram for “Timesheet Operation Flow”

The activity diagram in *Figure 9* models the timesheet logging workflow using 4 different swimlanes, them being *GeneralLabouerer*, *TimesheetLogin*, *Timesheet*, and *Entry*. Each swimlane has its own responsibility following the **Single Responsibility Principle**, by ensuring that no class handles all of the operations for this workflow. Because validation is a separate responsibility, *TimesheetLogin* was created just to handle that operation. *Timesheet* handles the business logic of this workflow, like loading the timesheet, and doing computations. Every swimlane is separated in their roles, and don't rely heavily on other swimlanes. Therefore this diagram showcases **low coupling** between classes, following a very important principle in software modelling.

The decision node in *TimesheetLogin* is very important for security reasons, as it decides whether or not the employee is allowed to access the timesheet at all. Having an outsider change your work details and hours of work could cause some issues later on, so it is very important for this to be secure and have validation. After validation, there is another decision node where the user can choose to submit a new entry, or get the pay summary. This node separates this workflow into two smaller workflows that happen based on the *GeneralLabourer*'s decision. The fork node in *Entry* showcases concurrency when computing two different calculations at the same time at once. This is to save time during calculations to improve the loading time for the *GeneralLabourer*. The join node ensures that both calculations are done before proceeding to the calculated hours join node, so that both the results are ready for the next computation. Each class has very focused and single responsibilities which demonstrates **high cohesion** in the classes e.g. *Entry* only records data, *TimesheetLogin* only handles credentials validation.

To further increase security of payments, there is a decision node in *Timesheet* which checks the hours calculated from *Entry*. The system needs to flag an issue if the *GeneralLabourer* entered the wrong details by accident or purpose. If this is not checked, the employee can put a fake number of hours, and be paid more on payday which would cause problems for the finance and management departments. When the hours validation fails, the *GeneralLabourer* is prompted to re-enter his details. All of the guard conditions mentioned in this section adheres to **Boehm's first law**, making sure any errors are caught early on (*Figure 8*).

2. Design Pattern Application and Implementation

2.1 Creational - Factory Design Pattern

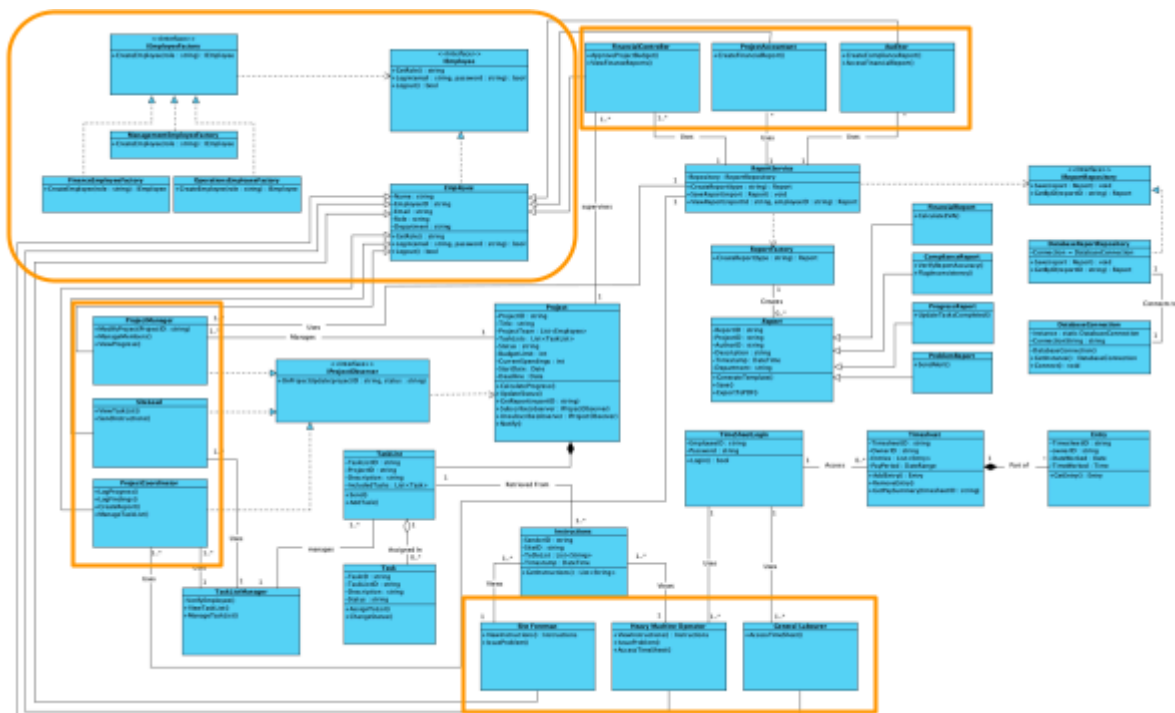


Figure 10: Refined Class Diagram with Factory Design Pattern Implementation

The Factory design pattern was applied to the creation of employees process using *IEmployeeFactory* as the interface for the 3 concrete implementations *ManagementEmployeeFactory*, *FinanceEmployeeFactory*, and *OperationsEmployeeFactory* (highlighted orange in *Figure 10*). Each of these three factories are responsible for instantiating new employees that belong in their respective departments e.g. *ProjectCoordinator* will be created by *ManagementEmployeeFactory*. This follows the **Single Responsibility Principle**, since the creation of employees are delegated to

the factories only and not throughout the system. This supports **low coupling**, separating creation logic from the rest of the system. Since *IEmployee* is an interface, each concrete class of *IEmployee* can change their code without affecting other classes, by implementing the interface.

During the creation of the refined class diagram, future-proofing was heavily considered, to ensure future changes can be done with ease. If a new department were to be added, a new concrete factory could implement *IFactory* without modifying existing factories, directly supporting the **Open/Closed Principle**. If the Factory method was absent from this diagram, employee creation would have been handled by large conditional statements or messy classes that are hard to maintain, and cause issues when multiple people are working on different employee creation behaviour.

```

src > Core > Interfaces > IEmployeeFactory.cs
1 namespace Core.Interfaces
2 {
3     // This interface defines the behaviour for all EmployeeFactories, where
4     public interface IEmployeeFactory
5     {
6         // This method creates and returns an employee based on the role given
7         IEmployee CreateEmployee(string role);
8     }
9 }

src > Factories > ManagementEmployeeFactory.cs
10 {
11     // This method creates and returns an employee from the management department
12     // Initialisation of "Project Manager", "Project Coordinator", "Site Lead"
13     public IEmployee CreateEmployee(string role)
14     {
15         switch (role.ToLower())
16         {
17             case "projectmanager":
18                 return new ProjectManager();
19             case "projectcoordinator":
20                 return new ProjectCoordinator();
21             case "sitelead":
22                 return new SiteLead();
23             default:
24                 // If role doesn't match any of the above -> return argument
25                 throw new ArgumentException($"Unknown management role: {role}");
26         }
27     }
28 }

src > Factories > FinanceEmployeeFactory.cs
10 {
11     // This method creates and returns an employee from the financial department
12     // Initialisation of "Financial Controller", "Project Accountant", and "Auditor"
13     public IEmployee CreateEmployee(string role)
14     {
15         switch (role.ToLower())
16         {
17             case "financialcontroller":
18                 return new FinancialController();
19             case "projectaccountant":
20                 return new ProjectAccountant();
21             case "auditor":
22                 return new Auditor();
23             default:
24                 // If role doesn't match any of the above -> return argument
25                 throw new ArgumentException($"Unknown finance role: {role}");
26         }
27     }
28 }

src > Factories > OperationsEmployeeFactory.cs
10 {
11     // This method creates and returns an employee from the operations department
12     // Initialisation of "Site Foreman", "Heavy Machine Operator", and "General Labourer"
13     public IEmployee CreateEmployee(string role)
14     {
15         switch (role.ToLower())
16         {
17             case "siteforeman":
18                 return new SiteForeman();
19             case "heavymachineoperator":
20                 return new HeavyMachineOperator();
21             case "generallabourer":
22                 return new GeneralLabourer();
23             default:
24                 // If role doesn't match any of the above -> return argument
25                 throw new ArgumentException($"Unknown operations role: {role}");
26         }
27     }
28 }

```

Figure 11: C# Implementation - Factory Design Pattern

The 4 files represent the Factory method implementation in the system program (Figure 11). The code written in the files was directly following the Factory Method in the refined class diagram (Figure 10). The *IEmployeeFactory* makes sure that all factories follow the same behaviour. In the concrete factories, the switch statement contains all creational logic to make sure no other class can directly create a new employee. By returning the *IEmployee* interface, the caller is only interacting with the abstraction, rather than the concrete details.

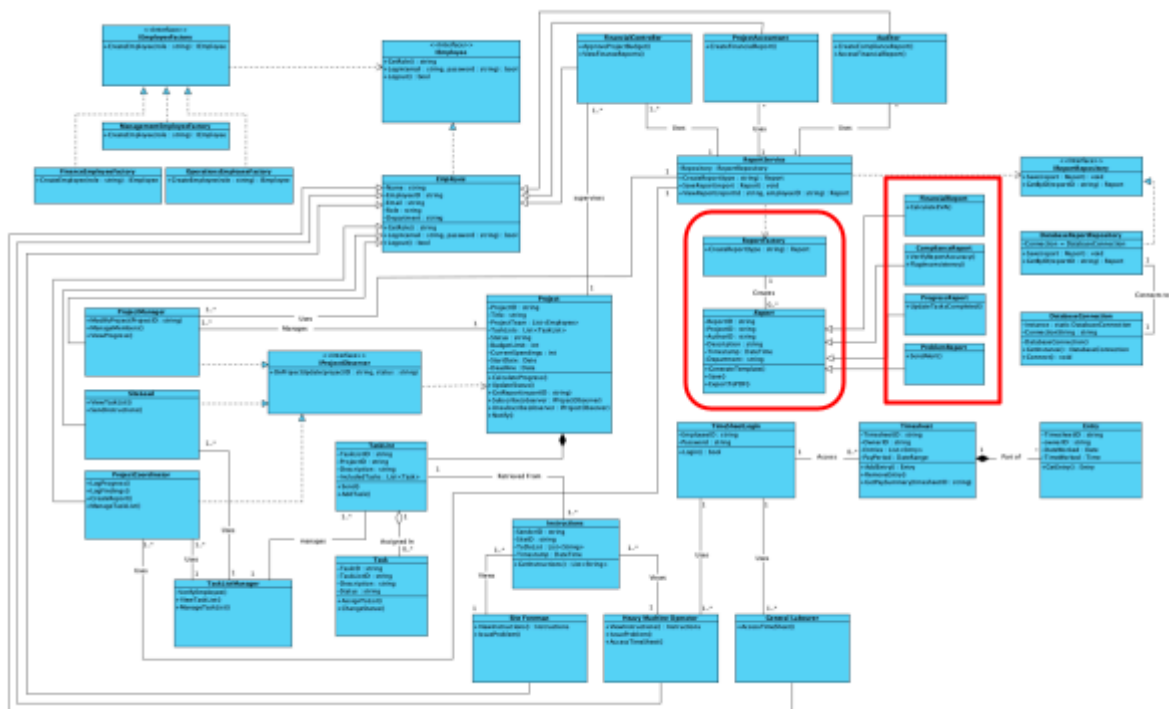


Figure 12: Refined Class Diagram with Simple Factory Method Implementation

A variant of the Factory design pattern called **Simple Factory** was also implemented in the refined class diagram (highlighted red in *Figure 12*). Factory method was more suitable for employee creation, however Simple factory seemed more suitable for report creation. The Simple Factory follows the **KISS Principle** - Keep It Super Simple. This was a design decision that was made during final review of a refined class diagram. In Simple Factory, there are no concrete implementations of an interface, instead there is a single class that contains all creation logic. This was deemed more suitable for this specific situation because reports are not split into different groups. Creating more factories would only cause a cluster of classes for something small like creating reports. Simple Factory doesn't follow Open/Closed Principle, however it does **avoid cluttering and creating unnecessary classes** on this scale.

2.1 Behavioural - Observer Design Pattern

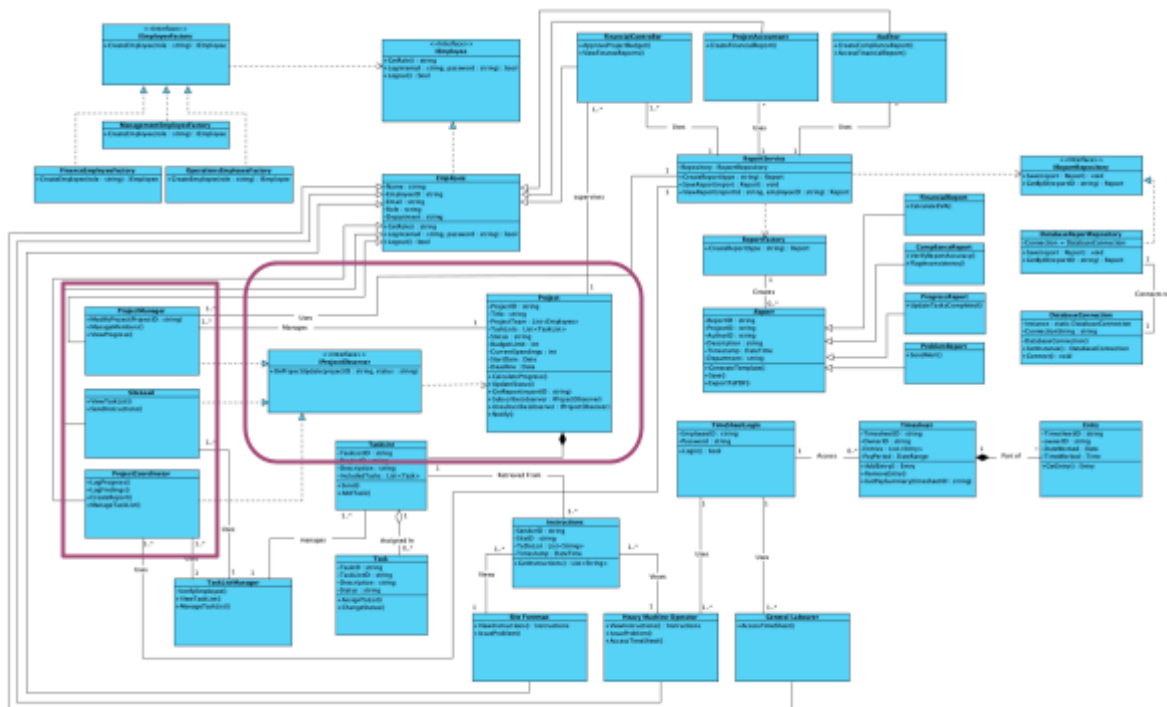


Figure 13: Refined Class Diagram with Observer Design Pattern Implementation

The Observer design pattern was applied to the management department, revolving around *Project* and the three roles in the management department *ProjectManager*, *ProjectCoordinatory*, and *SiteLead*. The *Project* class is the subject, containing the *Subscribe()*, *Unsubscribe()*, and *Notify()* methods which determine who gets project updates and who doesn't. When a project's status changes due to the *ProjectManager*'s action, *Notify()* is called, and sends updates to any employees who have been subscribed as an observer through the *OnProjectUpdate()* method implementation from *IPROjectObserver*. *Project* sends a notification, and each observer handles the response in their own implemented way. This directly showcases **high cohesion** as each class does only one thing.

The design of this section maintains **low coupling** between *Project* and the dependent classes. Without the Observer pattern, *Project* would need to hold references to all employees who need to receive updates. If more roles were to be added that needed to receive updates, *Project* would have to modify its list of references. However, with the Observer method, new employee roles can be added and implement *IPProjectObserver*, directly supporting the **Open/Closed Principle**. Therefore, making the notification design stable and scalable.

```

src > Core > Models > Project.cs
4
5 {
6
7 // Constructor that initialises a new project and saves it to the database.
8 // ProjectTeam, TaskLists, and Observers are initialised as empty lists - later
9 // added dynamically.
10 public Project(string projectId, string title, DateTime startDate, DateTime
11 deadline, int budgetLimit)
12 {
13     ProjectID = projectId;
14     Title = title;
15     StartDate = startDate;
16     Deadline = deadline;
17     BudgetLimit = budgetLimit;
18     CurrentSpending = 0;
19     Status = "Active";
20     ProjectTeam = new List<Employee>();
21     TaskLists = new List<TaskList>();
22     Observers = new List<IProjectObserver>();
23 }
24
25 // Adds an observer to the list so they can receive status updates
26 public void Subscribe(IProjectObserver observer)
27 {
28     Observers.Add(observer);
29 }
30
31 // Removes an observer from the list, so they no longer receive status updates
32 public void Unsubscribe(IProjectObserver observer)
33 {
34     Observers.Remove(observer);
35 }
36
37 // Notifies all observers when the status of this project changes - called
38 // automatically when status is changed
39 public void Notify()
40 {
41     foreach (IProjectObserver observer in Observers) {
42         // Calls OnProjectUpdate() for each observer in the list
43         observer.OnProjectUpdate(ProjectID, Status);
44     }
45 }
46
47 // Calculates the current progress depending on completed tasks and task lists
48 public void CalculateProgress()
49 {
50 }
51
52 }
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
262
```


3. Comparative Design Analysis

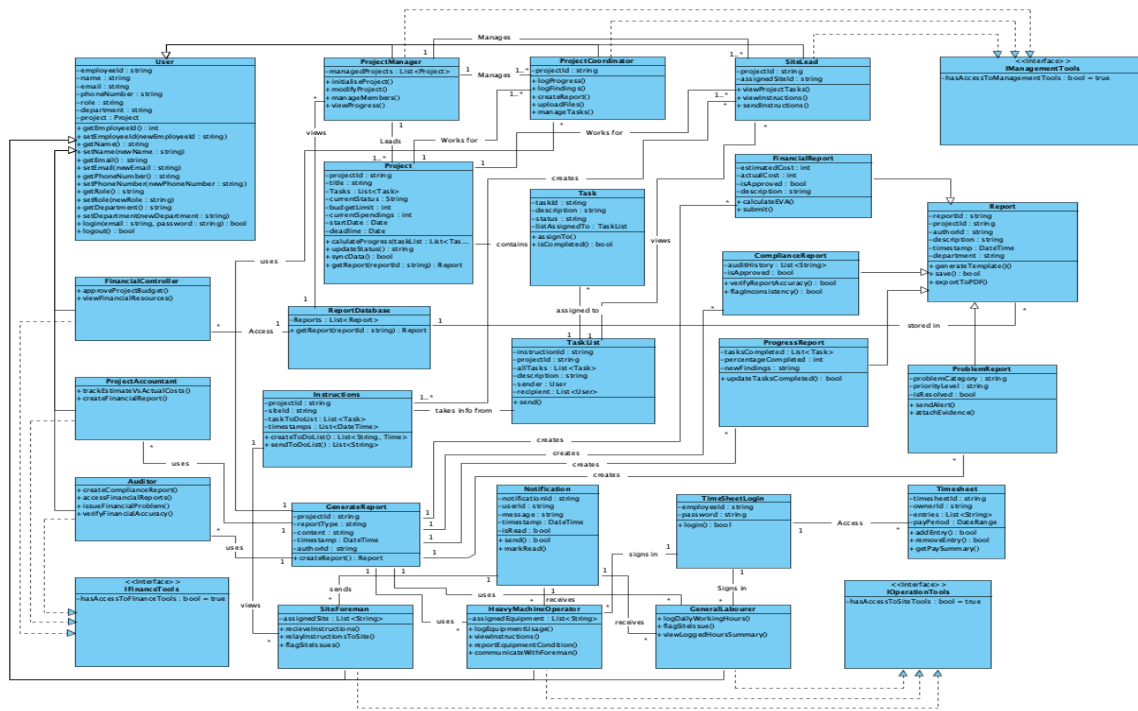


Figure 15: Legacy UML Class Diagram

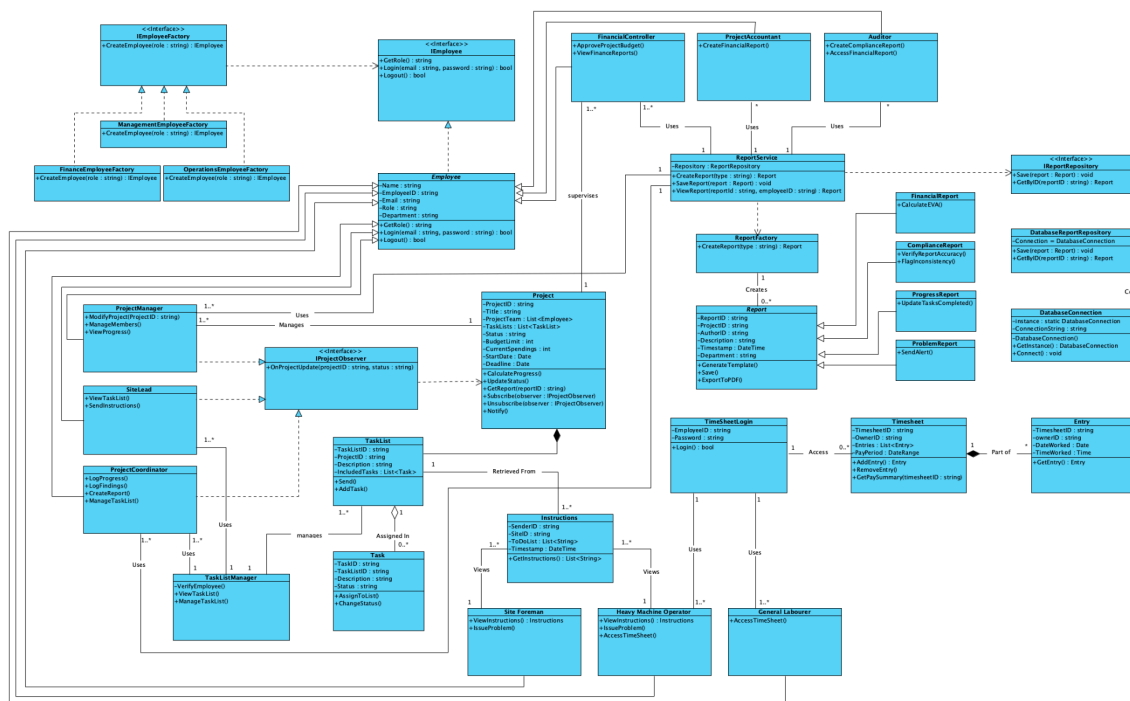


Figure 16: Refined UML Class Diagram

3.1 Changes to Class Structure and Responsibility Allocation

One of the biggest structural changes between the legacy (*Figure 15*) and refined (*Figure 16*) class diagrams is the breakdown of the monolithic *User* class. In the legacy diagram, there is a **monolithic** *User* class that handles everything about the user, including object creation, validation, business logic, and database entry. The *User* class contained methods like *SetDepartment()* and *GetPhoneNumber()* and attributes like *PhoneNumber* and *Project*. This meant that roles like *ProjectAccountant* and *GeneralLabourer* were represented by one class only having *Role* as a difference. Because the *User* class handled everything, it is **violating the Single Responsibility Principle** where every class should have one responsibility.

In the Refined class diagram, these problems were solved by breaking down the monolithic *User* class, and splitting it up into many classes responsible for different things. The **concrete** *User* class is **replaced by an abstract** *Employee* class that contains shared attributes taken from the *IEmployee* interface, which is what new concrete role classes implement. Role-specific behaviour is given to the separate concrete classes that implement *IEmployee*. The addition of *IEmployeeFactory* and the three concrete factories isolates creational logic from the business logic. This is because no class outside the three concrete classes is able to directly instantiate an *IEmployee* implementation. These changes solved the previous problems of having a monolithic entity, and followed the **Single Responsibility Principle**, while giving the classes **high cohesion**.

The second biggest change from the legacy class diagram to the refined class diagram was the complete restructuring of *Report*. In the legacy class diagram, *GenerateReport* was a concrete class that called *CreateReport()* in order to create a report object depending on the type inputted. This is not a good design because orchestration and creation is handled by one class which violates **Single Responsibility Principle**. There was also a missing structured creation mechanism, meaning any class could have created a report whenever they wanted. In the refined class diagram, all eligible employees interact with *ReportService* which handles all orchestration for reports. *ReportFactory* was the concrete class that handled report creation, which was only called by *ReportService*. *DatabaseReportRepository* was the concrete class that stored and retrieved reports to and from the report database.

3.2 Improvements on Cohesion & Reductions on Coupling

The **legacy class diagram** contained a **significant amount of coupling problems**. The monolithic *User* class represented every employee, only differentiating in *Role*. This is a problem because any class interacting with a specific role, has to make sure that that employee actually has that role during runtime. This created very high coupling because the behaviour for all the different roles were all based on the same class. In the refined class diagram, this was fixed by splitting all behaviour between the different *Employee* classes (the only shared behaviour was logging in and out of the system based) and all classes

implementing *IEmployee*. After these modifications, changes made to *ProjectManager* are completely isolated from *GeneralLabourer* and the other way around, thus achieving **low coupling**. This makes each class **highly cohesive**, and the system more stable.

The addition of the Observer pattern in the refined class diagram played a big role in reducing coupling within the system. In the legacy design, *ProjectManager*, *ProjectCoordinator*, and *SiteLead* were connected to the project through direct association with no stated notification system. In order to receive updates, *Project* would have to store references to each role and call their methods directly. This makes every project and their management department roles very highly coupled, because each time a new role needs to get notifications, *Project* needs to be modified. In the refined diagram, *Project* contains a private List of *IProjectObserver* and calls *OnProjectUpdate()* on each one. *Project* doesn't know who is listening to the update or how they react, **reducing the coupling to a minimum**. Since *Project* only handles notifications and project state, it's **cohesion is increased**.

The *Report* and *ReportDatabase* went through major changes that improved cohesion and decreased coupling. In the legacy system, *ReportDatabase* was an isolated class that stored a list of *Report* objects, and had a method *GetReport()* which could be called by anyone with the role of *ProjectManager*. In the refined class diagram, *ReportDatabase* is replaced with *DatabaseReportRepository* which implements the *IReportRepository* interface. There is a singleton *DatabaseConnection* class which handles the connection. This isolates each of the classes as *DatabaseReportRepository* is only focused on persistence, and *DatabaseConnection* is only focused on the actual connection, therefore **decreasing coupling**. Both of these classes only have one responsibility, therefore achieving **high cohesion**. The current *IReportRepository* interface further decouples *ReportService* from the concrete repository, which allows the storing method to be changed or swapped out without modifying the business logic.

Github Link:

<https://github.com/fatehbhular/FultonHogan-FatehBhular-23217987>